Área de Informática y Nuevas Tecnologías

Documento de Despliegue

SmartLining

SMTLNG

Nombre del fichero:	DAW_PWR_SMTLNG_5.pdf

Fecha de esta versión:	22/02/2026

Historial de revisiones

Fecha	Descripción	Autor
22/02/2026	Creación del documento de despliegue – Primera release	Aitor Peña Sánchez
27/02/2026	Ajustes finales y validación de scripts SQL (MVP)	Aitor Peña Sánchez

ÍNDICE

1 INTRODUCCIÓN 4
2 DETALLE DE LAS MEJORAS 4
2.1 Mejoras técnicas 4
2.2 Mejoras funcionales 4
3 SERVIDOR DE APLICACIONES 4
3.1 Identificación del servidor 4
3.2 Requisitos mínimos 4
4 SERVIDOR DE BASE DE DATOS 4
4.1 Identificación del servidor 4
4.2 Requisitos mínimos 4
4.3 Scripts 4
5 FICHEROS O BINARIOS A DESPLEGAR 4
5.1 Ubicación 4
6 OPERATIVA 5
6.1 Operaciones de base de datos 5
6.2 Operaciones de despliegue 5
6.3 Condiciones y verificación de éxito 5
7 CONTROL DE VERSIONES 5
8 OBSERVACIONES 5

1 INTRODUCCIÓN

El presente documento describe el proceso de despliegue del MVP del sistema **SmartLining**, una plataforma web de gestión de colas virtuales con analítica de afluencia.

Esta versión incluye:

- Gestión de colas mediante código QR.
- Gestión de turnos en tiempo real.
- Registro automático de tiempos de espera y atención.
- Panel de analítica con métricas clave.
- Sistema de actualización en tiempo real mediante SSE.
- Despliegue mediante Docker y docker-compose.

Se trata del primer despliegue en entorno reproducible.

2 DETALLE DE LAS MEJORAS

2.1 Mejoras técnicas

- Implementación de backend en Node.js + Express 5 + TypeScript.
- Integración de ORM Prisma v6 con MySQL 8.
- Contenerización completa con Docker multi-stage.
- Configuración mediante variables de entorno.
- Sistema de eventos en tiempo real mediante Server-Sent Events (SSE).
- Script de seed automático para generación de datos de prueba.
- Estructura modular y escalable del backend.
- Separación frontend/backend (React + Vite).

2.2 Mejoras funcionales

- Generación de turno sin registro mediante QR.
- Consulta en tiempo real del estado del turno.
- Gestión de turnos por parte del empleado.
- Panel de administración con métricas:
    - Tiempo medio de espera.
    - Tiempo medio de atención.
    - Clientes por franja horaria.

3 SERVIDOR DE APLICACIONES

3.1 Identificación del servidor

- Nombre del servidor en la red: smartlining-app
- Dominio: (según entorno de despliegue, ej. smartlining.local o dominio productivo)
- Directorio raíz del host virtual: `/app` (contenedor Docker)

3.2 Requisitos mínimos

- Servidor de aplicaciones: Node.js
- Versión mínima: Node.js v18 (recomendado v23)
- Otros:
    - npm
    - Docker y docker-compose (recomendado)
    - Acceso a variables de entorno (.env)

3.3 Configuración recomendada

- Memoria recomendada: mínimo 512MB RAM (recomendado 1GB).
- Uso de HTTPS en producción mediante reverse proxy (Nginx recomendado).
- Configuración CORS correctamente definida.

4 SERVIDOR DE BASE DE DATOS

4.1 Identificación del servidor

- Nombre del servidor en la red: smartlining-db
- Esquema/base de datos: smartlining

4.2 Requisitos mínimos

- Motor: MySQL
- Versión mínima: MySQL 8.0.x
- Otros:
    - Usuario con permisos SELECT, INSERT, UPDATE, DELETE.
    - Permisos adicionales si se ejecutan migraciones.

4.3 Scripts

El operador deberá ejecutar:

- **docker-compose up -d --build:** Desde la raíz del repositorio

  Esto levantará el servicio `db` (MySQL 8) y `app` (imagen multi-stage)

  `El contenedor `db` expone el puerto `3307` en la máquina host (mapeado a 3306 interno por `docker-compose.yml`).`

5 FICHEROS O BINARIOS A DESPLEGAR

5.1 Ubicación

El código fuente se entrega mediante repositorio GitHub:

Repositorio remoto:
https://github.com/[usuario]/smartlining

6 OPERATIVA

6.1 Operaciones de base de datos

1. npm run db:migrate # desarrollo con shadow DB (requiere CREATE DATABASE)
2. npm run db:push # crea/actualiza esquema sin migraciones
3. npm run db:seed # inyecta datos en la bbdd

6.2 Operaciones de despliegue

1. git clone https://github.com/[usuario]/smartlining.git
2. Crear archivo .env basado en .env.example.
3. docker-compose up -d --build

6.3 Condiciones y verificación de éxito

1. Acceder a la aplicación web desde navegador.
2. Comprobar login

   POST /api/login
3. Generar un turno y verificar que:
   - Se guarda en base de datos.
   - Se actualiza el estado correctamente.
4. Comprobar que el dashboard muestra métricas.
5. Verificar logs sin errores críticos:

   docker-compose logs -f app

Si todas las comprobaciones son correctas, el despliegue se considera exitoso.

7 CONTROL DE VERSIONES

https://github.com/[usuario]/smartlining

La versión desplegada corresponde a: *main*

8 OBSERVACIONES

- No se deben subir credenciales reales al repositorio.
- El archivo `.env` no debe incluirse en control de versiones.
- Se recomienda desplegar bajo HTTPS en producción.
- Para futuras versiones se recomienda implementar pipeline CI/CD.
- La arquitectura es escalable y permite ampliaciones sin modificar la base del sistema.